



INSTITUTO POLITÉCNICO NACIONAL
UNIDAD PROFESIONAL INTERDISCIPLINARIA
DE CIENCIAS SOLIALES Y ADMINISTRATIVAS

Lic. Ciencias de la Informática

Semestre: Tercero

Secuencia: 3CM30

Maestro: Yventz Entzana Garduño

Materia: Abstracción y Uso de Datos

Integrantes:

- NL 10 García Galicia Frida Abigail.
- NL 38 Saúl León Ángel Alexander.

Título del Proyecto:

Prácticas 3er Parcial.

Fecha de Entrega:

18-Junio-2026

Link del repositorio:

https://github.com/AlexxanderSaul/Abstraccion_y_uso_de_Datos

REPORTE INFORMATIVO

Práctica Datos Puntero Arreglo

Concepto General

En este proyecto desarrollamos un sistema en C++ para el registro y almacenamiento de información personal utilizando arreglos dinámicos y punteros. El programa permite agregar datos de personas, mostrarlos en pantalla y exportarlos a diferentes formatos de archivo para facilitar su consulta y utilización en otros sistemas.

Durante el desarrollo se aplicaron conceptos relacionados con programación orientada a objetos, manejo de memoria dinámica, uso de punteros, estructuras de datos y generación de archivos en formatos ampliamente utilizados como TXT, CSV, XML y JSON.

Definición

El programa consiste en una clase denominada DatosPunteroArreglo, cuya función principal es administrar una colección de registros de personas mediante un arreglo dinámico.

Cada persona almacena información básica como:

- Nombre
- Apellido paterno
- Apellido materno
- Género
- Edad

Los registros son almacenados en memoria utilizando punteros y posteriormente pueden visualizarse o guardarse en diferentes formatos de archivo.

Objetivo

Desarrollar una aplicación que permita registrar información de personas utilizando arreglos dinámicos y punteros, facilitando su almacenamiento y exportación en múltiples formatos de archivo.

- Aplicar el uso de punteros y memoria dinámica en C++.
- Implementar una estructura para almacenar información personal.
- Permitir el registro de múltiples personas.
- Mostrar los datos almacenados en pantalla.
- Validar que el arreglo no se encuentre lleno o vacío.
- Generar archivos TXT, CSV, XML y JSON.
- Aplicar técnicas de validación y organización de información.

Proceso de elaboración

El proyecto fue realizado en pareja, por lo que las actividades fueron distribuidas para facilitar el desarrollo y la integración de cada componente.

1. Creación de la estructura de almacenamiento

Se diseñó una estructura llamada Persona, la cual contiene los atributos necesarios para almacenar la información de cada individuo:

- Nombre
- Apellido paterno
- Apellido materno
- Género
- Edad

Esta estructura sirve como base para la gestión de los registros.

2. Implementación del arreglo dinámico

Se creó un arreglo dinámico utilizando memoria reservada mediante:

```
personas = new Persona[MAX];
```

Con esta instrucción se genera el espacio necesario para almacenar múltiples registros de personas.

Asimismo, se implementó el destructor correspondiente para liberar correctamente la memoria utilizada:

```
delete[] personas;
```

Esto evita fugas de memoria durante la ejecución del programa.

3. Registro de personas

Se desarrolló la función agregarPersona() para insertar nuevos registros dentro del arreglo dinámico.

Antes de agregar un nuevo elemento, el sistema verifica si aún existe espacio disponible. En caso contrario, muestra el mensaje:

Lista llena

Si existe capacidad disponible, el registro es almacenado mediante aritmética de punteros.

4. Visualización de registros

La función mostrarPersonas() permite recorrer el arreglo y mostrar toda la información registrada.

Para cada persona se presentan:

- Nombre completo
- Género
- Edad

Además, se valida que existan registros antes de intentar mostrarlos.

5. Validación del estado de la lista

Se implementaron funciones auxiliares para verificar la condición del arreglo:

`vacio()`

Determina si no existen registros almacenados.

`lleno()`

Verifica si se alcanzó la capacidad máxima permitida.

`tamano()`

Retorna la cantidad actual de registros almacenados.

Estas funciones permiten un mejor control del sistema y evitan errores durante la ejecución.

6. Exportación de archivos

La función `guardarArchivos()` permite generar diferentes tipos de archivos a partir de los datos registrados.

Archivo TXT

Almacena la información en formato de texto simple, colocando cada registro en una línea independiente.

Archivo CSV

Genera una tabla separada por comas compatible con aplicaciones como Excel y Google Sheets.

Archivo XML

Organiza la información mediante etiquetas jerárquicas, facilitando el intercambio de datos entre sistemas.

Archivo JSON

Almacena la información utilizando objetos y arreglos compatibles con aplicaciones web y bases de datos modernas.

7. Manejo de caracteres especiales

Para evitar errores durante la generación de archivos estructurados se implementaron dos funciones especiales:

`escJson()`

Se encarga de reemplazar caracteres especiales para que los archivos JSON mantengan una sintaxis válida.

`escXml()`

Convierte caracteres reservados de XML en entidades seguras para evitar errores de interpretación.

Estas funciones mejoran la compatibilidad y confiabilidad de los archivos generados.

Comentario

Durante el desarrollo del proyecto se realizó una mejora en la generación del archivo XML agregando la declaración estándar al inicio del documento:

```
xml << "<?xml version=\"1.0\" encoding=\"UTF-8\" ?>\n";
```

Esta modificación permite que los archivos XML cumplan con los estándares internacionales de codificación y estructura, facilitando su lectura por diferentes aplicaciones y sistemas.

Además, se implementaron funciones de escape para XML y JSON, lo que garantiza que caracteres especiales dentro de los nombres o apellidos no provoquen errores al momento de generar los archivos.

Conclusión

Con este proyecto logramos aplicar correctamente el uso de punteros y arreglos dinámicos para el almacenamiento de información personal. Asimismo, reforzamos conocimientos sobre memoria dinámica, validación de datos, estructuras, manejo de archivos y exportación de información en múltiples formatos. El trabajo colaborativo permitió organizar mejor las tareas y obtener una solución funcional, eficiente y compatible con diferentes sistemas de almacenamiento e intercambio de datos.

REPORTE INFORMATIVO

Práctica Tamaño Datos

Concepto General

En este proyecto desarrollamos un programa en C++ que permite identificar y mostrar el tamaño en memoria de los principales tipos de datos básicos utilizados en el lenguaje. Además de visualizar esta información en pantalla, el sistema genera archivos en diferentes formatos para almacenar los resultados obtenidos.

Durante la elaboración del proyecto se aplicaron conocimientos relacionados con tipos de datos, manejo de archivos, programación orientada a objetos y conversión de información a formatos estructurados como XML y JSON.

Definición

Los tipos de datos son elementos fundamentales en programación, ya que permiten definir qué tipo de información puede almacenarse dentro de una variable y cuánto espacio ocupará en memoria.

El operador `sizeof()` de C++ permite conocer la cantidad de bytes que utiliza cada tipo de dato dentro de la arquitectura donde se ejecuta el programa.

Este proyecto utiliza dicho operador para obtener el tamaño de los ocho tipos de datos básicos más comunes y presentar la información de manera organizada.

Objetivo

Desarrollar una aplicación que permita consultar y almacenar el tamaño de los principales tipos de datos básicos de C++ en bytes y bits.

Objetivos Específicos

- Utilizar el operador `sizeof()` para obtener información de memoria.
- Mostrar el tamaño de diferentes tipos de datos básicos.
- Comprender la relación entre bytes y bits.
- Generar archivos TXT, CSV, XML y JSON con la información obtenida.
- Aplicar conceptos de programación orientada a objetos.
- Practicar el manejo de archivos en diferentes formatos.

Proceso de elaboración

El proyecto fue desarrollado en pareja, distribuyendo las tareas de análisis, programación y pruebas para lograr una mejor organización del trabajo.

1. Creación de la clase TamaniosDatos

Se diseñó la clase TamaniosDatos, encargada de almacenar y gestionar toda la información relacionada con los tamaños de los tipos de datos.

Dentro de la clase se implementó una variable denominada contenido, la cual almacena el texto generado para posteriormente mostrarlo o guardarlo en archivos.

2. Implementación de la función genérica

Para evitar repetir código se desarrolló una función plantilla:

```
template<typename T>
void agregarTipo(ostringstream& os, const char nombre[])
```

Esta función recibe un tipo de dato y utiliza el operador sizeof() para calcular:

- Cantidad de bytes.
- Cantidad de bits.

Los resultados son agregados automáticamente al flujo de salida.

3. Obtención de los tamaños de datos

Se implementó la función mostrarBasicos(), encargada de consultar los ocho tipos de datos básicos:

- char
- short
- int
- long
- long long
- float
- double
- bool

Para cada uno se calcula el espacio que ocupa en memoria y posteriormente se almacena la información generada.

4. Validación de información

Se creó la función vacio() para verificar si la variable que almacena los resultados contiene información.

Esta validación permite evitar errores al momento de generar los archivos y garantiza que siempre exista contenido disponible para guardar.

5. Generación de archivos

La función `guardarArchivos()` permite exportar la información obtenida en cuatro formatos diferentes.

Archivo TXT

Almacena la información exactamente como se muestra en pantalla.

Archivo CSV

Guarda los datos en formato tabular utilizando columnas para:

- Tipo de dato.
- Bytes.
- Bits.

Este formato puede abrirse fácilmente en programas como Excel.

Archivo XML

Organiza la información mediante etiquetas estructuradas que facilitan el intercambio de datos entre aplicaciones.

Archivo JSON

Almacena los datos utilizando objetos y arreglos compatibles con aplicaciones web y sistemas modernos.

6. Conversión de caracteres especiales

Durante el desarrollo también se implementaron funciones auxiliares:

`escJson()`

Permite convertir caracteres especiales para mantener la sintaxis correcta en archivos JSON.

`escXml()`

Convierte caracteres reservados de XML en entidades seguras para evitar errores de interpretación.

Aunque en este proyecto los nombres de los tipos de datos no contienen caracteres especiales, estas funciones fueron incorporadas para mejorar la robustez y reutilización del código.

Comentario

Durante el desarrollo del proyecto se realizó una mejora en la generación del archivo XML agregando la declaración estándar al inicio del documento:

```
xml << "<?xml version=\"1.0\" encoding=\"UTF-8\" ?>\n";
```

Esta línea permite identificar correctamente la versión de XML utilizada y la codificación de caracteres del documento.

La incorporación de esta declaración mejora la compatibilidad del archivo con diferentes sistemas, editores y herramientas de procesamiento de datos.

Además, el proyecto permitió comprender de forma práctica cómo varía el espacio de almacenamiento que utilizan los distintos tipos de datos dentro de un programa, información fundamental para optimizar el uso de memoria en aplicaciones futuras.

Conclusión

Con este proyecto logramos comprender la importancia de los tipos de datos y el espacio que ocupan en memoria dentro del lenguaje C++. También reforzamos conocimientos sobre el uso del operador sizeof(), la programación orientada a objetos y la generación de archivos en múltiples formatos. El trabajo colaborativo facilitó el desarrollo de una solución organizada que permite consultar, almacenar y analizar información relacionada con la administración de memoria en programación.

REPORTE INFORMATIVO

Práctica Matrices

Concepto General

En este proyecto desarrollamos una aplicación en C++ para el manejo de matrices. El programa permite ingresar dos matrices, visualizarlas, realizar operaciones matemáticas como la multiplicación por una constante y la multiplicación entre matrices, además de guardar la información en distintos formatos de archivo.

Durante la elaboración del proyecto se aplicaron conceptos fundamentales de álgebra matricial, estructuras bidimensionales, validación de datos, programación orientada a objetos y manejo de archivos.

Definición

Una matriz es una estructura de datos bidimensional formada por filas y columnas, utilizada para representar información numérica de manera organizada.

Las matrices tienen una amplia aplicación en áreas como matemáticas, física, inteligencia artificial, gráficos por computadora y análisis de datos.

Este programa permite trabajar con dos matrices denominadas A y B, realizando operaciones básicas y almacenando la información generada en diferentes formatos para su posterior utilización.

Objetivo

Desarrollar una aplicación que permita ingresar, visualizar, operar y almacenar matrices utilizando el lenguaje de programación C++.

- Implementar matrices bidimensionales utilizando arreglos.
- Permitir el ingreso de datos por parte del usuario.
- Validar las dimensiones de las matrices.
- Realizar multiplicación de una matriz por una constante.
- Implementar la multiplicación entre matrices.
- Exportar la información a formatos TXT, CSV, XML y JSON.
- Aplicar conceptos de programación orientada a objetos y manejo de archivos.

Proceso de elaboración

El proyecto fue realizado en pareja, distribuyendo las actividades de análisis, diseño, programación y pruebas para lograr una implementación más organizada y eficiente.

1. Diseño de la clase Matrices

Inicialmente se creó la clase Matrices, encargada de administrar toda la información relacionada con las matrices.

Se definieron variables para almacenar:

- Número de filas de la matriz A.
- Número de columnas de la matriz A.
- Número de filas de la matriz B.
- Número de columnas de la matriz B.

También se incorporaron variables de control para verificar si cada matriz había sido capturada correctamente.

2. Captura de matrices

Se desarrollaron las funciones:

- leerMatrizA()
- leerMatrizB()

Estas funciones permiten solicitar al usuario las dimensiones de cada matriz y posteriormente ingresar los valores correspondientes.

Se implementaron validaciones para asegurar que las dimensiones se encuentren dentro del rango permitido, el cual es de hasta 10 filas y 10 columnas.

Cuando el usuario introduce dimensiones inválidas, el sistema muestra un mensaje de error y evita continuar con el proceso.

3. Visualización de matrices

Se implementaron las funciones:

- mostrarMatrizA()
- mostrarMatrizB()

Estas funciones recorren las matrices utilizando ciclos anidados y muestran los elementos organizados en filas y columnas.

Además, se verifica previamente que las matrices hayan sido capturadas antes de intentar visualizarlas.

4. Multiplicación por una constante

Se desarrolló la función multiplicarPorConstante().

Esta operación consiste en multiplicar cada elemento de la matriz A por un valor numérico ingresado por el usuario.

Para realizar el cálculo, el programa recorre toda la matriz y multiplica cada posición por la constante indicada, mostrando el resultado en pantalla.

Esta operación es ampliamente utilizada en álgebra lineal para transformar matrices mediante factores de escala.

5. Multiplicación entre matrices

Se implementó la función `multiplicarMatrices()`, encargada de realizar la multiplicación de la matriz A por la matriz B.

Antes de efectuar la operación, el programa verifica la condición matemática necesaria para que la multiplicación sea posible:

- El número de columnas de la matriz A debe ser igual al número de filas de la matriz B.

Si esta condición no se cumple, el sistema informa el error y evita realizar cálculos incorrectos.

Cuando las dimensiones son compatibles, se utiliza una tercera matriz auxiliar para almacenar los resultados obtenidos mediante la fórmula de multiplicación matricial.

6. Verificación de existencia de matrices

Se desarrollaron las funciones:

- `existeA()`
- `existeB()`

Estas funciones permiten verificar si las matrices fueron ingresadas correctamente.

Gracias a estas validaciones se evita ejecutar operaciones sobre matrices inexistentes.

7. Generación de archivos

La función `guardarArchivos()` permite almacenar la información capturada en cuatro formatos diferentes.

Archivo TXT

Guarda las matrices en formato de texto simple conservando la estructura de filas y columnas.

Archivo CSV

Almacena cada elemento indicando:

- Matriz de origen.
- Fila.
- Columna.
- Valor.

Este formato facilita la importación de datos a hojas de cálculo.

Archivo XML

Representa las matrices mediante etiquetas estructuradas que describen sus dimensiones y valores.

Archivo JSON

Almacena las matrices utilizando arreglos bidimensionales compatibles con aplicaciones web y sistemas modernos.

Comentario

Durante el desarrollo del proyecto se realizó una mejora en la generación del archivo XML agregando la declaración estándar al inicio del documento:

```
xml << "<?xml version=\"1.0\" encoding=\"UTF-8\" ?>\n";
```

Esta línea permite identificar correctamente la versión del lenguaje XML y la codificación utilizada para almacenar los caracteres.

La incorporación de esta declaración mejora la compatibilidad de los archivos generados con diferentes programas de edición, procesamiento e intercambio de datos.

Además, se implementaron diversas validaciones para garantizar que las operaciones matriciales se realicen únicamente cuando las dimensiones sean correctas, evitando errores matemáticos durante la ejecución del programa.

Conclusión

Con este proyecto logramos aplicar de manera práctica los conceptos relacionados con matrices y operaciones matriciales dentro del lenguaje C++. También reforzamos conocimientos sobre arreglos bidimensionales, validación de datos, estructuras de control y manejo de archivos en distintos formatos. El trabajo realizado en equipo permitió desarrollar una solución organizada y funcional, capaz de gestionar matrices y almacenar la información de forma eficiente para su uso posterior.

REPORTE INFORMATIVO

Práctica Promedio puntero

Concepto General

En este proyecto desarrollamos una aplicación en C++ que permite capturar cinco valores numéricos utilizando memoria dinámica y punteros. A partir de estos datos, el programa calcula diferentes medidas estadísticas básicas como la suma, el promedio, el valor máximo y el valor mínimo.

Además, la información procesada puede almacenarse en distintos formatos de archivo para facilitar su consulta y utilización posterior. Durante el desarrollo se aplicaron conceptos relacionados con punteros, memoria dinámica, operaciones matemáticas, programación orientada a objetos y manejo de archivos.

Definición

Los punteros son variables especiales que almacenan direcciones de memoria y permiten acceder de forma directa a los datos almacenados en ella.

En este proyecto se utiliza un arreglo dinámico de tipo double para almacenar cinco números ingresados por el usuario. Posteriormente, mediante operaciones realizadas con punteros, se calculan distintos indicadores estadísticos que permiten analizar los datos capturados.

El programa también genera archivos en formatos TXT, CSV, XML y JSON para almacenar los resultados obtenidos.

Objetivo

Desarrollar una aplicación que permita capturar datos numéricos mediante punteros y calcular medidas estadísticas básicas para posteriormente almacenarlas en diferentes formatos de archivo.

- Aplicar el uso de punteros y memoria dinámica.
- Capturar datos numéricos proporcionados por el usuario.
- Calcular la suma de los valores almacenados.
- Obtener el promedio o media aritmética.
- Identificar el valor máximo y mínimo del conjunto de datos.
- Generar archivos TXT, CSV, XML y JSON.
- Aplicar conceptos de programación orientada a objetos.

Proceso de elaboración

El proyecto fue realizado en pareja, distribuyendo las actividades de diseño, programación, validación y pruebas para obtener una solución funcional y organizada.

1. Creación de la clase PromedioPuntero

Inicialmente se diseñó la clase PromedioPuntero, encargada de administrar los datos numéricos y realizar todos los cálculos estadísticos.

Dentro del constructor se reservó memoria dinámica para almacenar cinco números reales utilizando:

```
datos = new double[tam];
```

Asimismo, se inicializó una variable de control que permite verificar si los datos ya fueron capturados.

2. Liberación de memoria

Para evitar fugas de memoria se implementó el destructor de la clase:

```
delete[] datos;
```

Esta instrucción libera correctamente el espacio reservado durante la ejecución del programa.

3. Captura de datos

Se desarrolló la función leerDatos(), encargada de solicitar al usuario cinco valores numéricos.

Los datos son almacenados utilizando aritmética de punteros mediante la expresión:

```
*(datos + i)
```

Una vez finalizada la captura, el programa marca que los datos ya se encuentran disponibles para realizar operaciones.

4. Visualización de información

La función mostrarDatos() permite mostrar en pantalla todos los valores almacenados.

Antes de realizar la visualización, el programa verifica que los datos hayan sido ingresados correctamente para evitar errores durante la ejecución.

5. Cálculo de la suma

Se implementó la función suma(), cuya finalidad es recorrer todos los elementos almacenados y acumular su valor.

El resultado obtenido representa la suma total de los números ingresados.

6. Cálculo del promedio

La función promedio() calcula la media aritmética dividiendo la suma total entre la cantidad de datos almacenados.

La función media() devuelve exactamente el mismo resultado, ya que ambos conceptos representan la misma medida estadística en este proyecto.

7. Obtención del valor máximo

Se desarrolló la función maximo(), encargada de identificar el número más grande dentro del conjunto de datos.

Para ello se compara cada elemento con el valor mayor encontrado hasta ese momento.

8. Obtención del valor mínimo

Se implementó la función minimo(), cuya finalidad es encontrar el número más pequeño almacenado en el arreglo.

El proceso consiste en comparar cada dato con el menor valor registrado durante el recorrido.

9. Validación de datos

Se creó la función estaVacio(), la cual permite verificar si los números ya fueron ingresados.

Esta validación evita que se realicen cálculos sobre datos inexistentes y mejora la estabilidad del programa.

10. Generación de archivos

La función guardarArchivos() permite almacenar los datos y resultados obtenidos en diferentes formatos.

Archivo TXT

Guarda los números capturados junto con la suma, promedio, máximo y mínimo.

Archivo CSV

Organiza los datos en columnas para facilitar su apertura en hojas de cálculo como Excel.

Archivo XML

Representa la información mediante etiquetas estructuradas que describen cada dato y los resultados calculados.

Archivo JSON

Almacena los valores en formato de objetos y arreglos compatibles con aplicaciones web y sistemas modernos.

Comentario

Durante el desarrollo del proyecto se realizó una mejora en la generación del archivo XML agregando la declaración estándar al inicio del documento:

```
xml << "<?xml version=\"1.0\" encoding=\"UTF-8\" ?>\n";
```

Esta línea permite identificar la versión del lenguaje XML utilizada y la codificación de caracteres del archivo.

Gracias a esta modificación, los documentos XML generados presentan una mejor compatibilidad con diferentes sistemas, navegadores y herramientas de procesamiento de datos.

Además, el proyecto permitió reforzar el uso práctico de punteros para acceder a la memoria dinámica y realizar operaciones estadísticas sin utilizar arreglos convencionales.

Conclusión

Con este proyecto logramos aplicar de manera práctica conceptos fundamentales de programación como punteros, memoria dinámica y manejo de arreglos. También reforzamos conocimientos relacionados con operaciones estadísticas básicas, validación de datos y generación de archivos en distintos formatos. El trabajo realizado en equipo permitió desarrollar una solución organizada y funcional que facilita el análisis de información numérica y el almacenamiento de resultados para su consulta posterior.

REPORTE INFORMATIVO

Práctica Triángulos

Concepto General

En este proyecto desarrollamos una aplicación en C++ capaz de generar dos fractales matemáticos conocidos: el Triángulo de Sierpinski y el Polvo de Cantor. Ambos fractales se construyen mediante procesos recursivos, lo que permite observar cómo una figura simple puede generar patrones complejos a partir de la repetición de una misma regla.

Además de mostrar los fractales en pantalla, el programa almacena la información generada en archivos TXT, CSV, XML y JSON, permitiendo conservar los resultados para su posterior análisis o consulta.

Durante el desarrollo se aplicaron conceptos de recursividad, estructuras dinámicas, programación orientada a objetos, manejo de cadenas y generación de archivos estructurados.

Definición

Un fractal es una figura geométrica que presenta patrones repetitivos en diferentes escalas. Estas estructuras son utilizadas en matemáticas, informática, física, diseño gráfico y simulación de fenómenos naturales.

En este proyecto se implementaron dos fractales:

Triángulo de Sierpinski

Es un fractal formado mediante la subdivisión repetitiva de triángulos. Cada nivel genera una estructura más compleja manteniendo el mismo patrón geométrico.

Polvo de Cantor

Es un fractal obtenido al dividir un segmento en tres partes iguales y eliminar continuamente la sección central. Este proceso se repite recursivamente en los segmentos restantes.

Objetivo

Desarrollar una aplicación que permita generar fractales mediante algoritmos recursivos y almacenar los resultados en diferentes formatos de archivo.

Objetivos Específicos

- Aplicar el concepto de recursividad en la construcción de fractales.
- Generar el Triángulo de Sierpinski.
- Generar el Polvo de Cantor.
- Almacenar la representación textual de los fractales.

- Exportar información en formatos TXT, CSV, XML y JSON.
- Implementar técnicas de validación y procesamiento de cadenas.
- Reforzar el uso de programación orientada a objetos.

Proceso de elaboración

El proyecto fue desarrollado en pareja, distribuyendo las tareas de análisis, programación, pruebas y documentación para lograr una solución organizada y funcional.

1. Creación de funciones de conversión de caracteres

Inicialmente se desarrollaron funciones auxiliares para evitar problemas al generar archivos estructurados.

`escaparJson()`

Permite reemplazar caracteres especiales que podrían generar errores en archivos JSON.

`escaparXml()`

Convierte caracteres reservados de XML en entidades válidas para mantener una estructura correcta.

`escaparCsv()`

Gestiona adecuadamente las comillas y caracteres especiales dentro de archivos CSV.

Estas funciones mejoran la compatibilidad y confiabilidad de los archivos generados.

2. Implementación del almacenamiento de datos

Se desarrolló la función `guardarArchivos()`, encargada de exportar la información obtenida a diferentes formatos.

Los datos almacenados incluyen:

- Nombre del programa.
- Tipo de fractal.
- Nivel seleccionado.
- Representación gráfica generada.

La información puede guardarse en:

- TXT
- CSV
- XML

- JSON

3. Desarrollo de la clase Sierpinski

Se implementó la clase Sierpinski, responsable de generar el Triángulo de Sierpinski.

Dentro del constructor se inicializan las variables necesarias para almacenar:

- Nivel del fractal.
- Tamaño de la matriz.
- Dibujo generado.

Posteriormente se solicita al usuario el nivel de profundidad que desea visualizar.

4. Construcción del Triángulo de Sierpinski

La generación del fractal se realiza mediante la función recursiva:

dibujar(int fila, int col, int n, int nivelActual)

Cuando el nivel llega a cero se coloca un carácter *, que representa la unidad mínima del fractal.

En niveles superiores, el algoritmo divide el problema en tres subtriángulos más pequeños:

- Triángulo superior.
- Triángulo inferior izquierdo.
- Triángulo inferior derecho.

Este proceso continúa hasta completar la figura correspondiente al nivel seleccionado.

5. Desarrollo de la clase Cantor

Se implementó la clase Cantor, encargada de generar el fractal conocido como Polvo de Cantor.

El usuario selecciona el nivel de profundidad y el programa calcula automáticamente la longitud necesaria para representar el fractal.

La figura inicial se construye utilizando una línea compuesta por caracteres -.

6. Construcción del Polvo de Cantor

La generación del fractal se realiza mediante la función recursiva:

dibujar(vector<char> linea, int nivelActual)

En cada llamada recursiva:

1. Se muestra la línea actual.
2. Se divide en tres partes iguales.
3. Se elimina visualmente la sección central.
4. Se continúa trabajando únicamente con los segmentos izquierdo y derecho.

Este proceso se repite hasta alcanzar el nivel indicado por el usuario.

7. Obtención de información del fractal

Se implementaron las funciones:

- obtenerDatos() para Sierpinski.
- obtenerDatos() para Cantor.

Estas funciones recopilan toda la información generada durante la ejecución y la preparan para su almacenamiento en los diferentes formatos disponibles.

Comentario

Durante el desarrollo del proyecto se realizó una mejora en la generación de archivos XML agregando la declaración estándar al inicio del documento:

```
xml << "<?xml version=\"1.0\" encoding=\"UTF-8\" ?>\n";
```

Esta modificación permite que los archivos XML cumplan con los estándares internacionales de codificación y estructura, facilitando su interpretación por diferentes programas y sistemas.

Asimismo, se incorporaron funciones de escape para XML, JSON y CSV, lo que garantiza que caracteres especiales presentes en los datos no provoquen errores durante la generación de los archivos.

El proyecto permitió comprender de forma práctica la aplicación de algoritmos recursivos en la construcción de fractales, mostrando cómo reglas simples pueden producir estructuras geométricas complejas.

Conclusión

Con este proyecto logramos implementar correctamente dos fractales matemáticos utilizando técnicas de recursividad en C++. Además de reforzar conocimientos sobre programación orientada a objetos y manejo de archivos, comprendimos la importancia de la división de problemas complejos en subproblemas más pequeños para generar patrones repetitivos. El trabajo realizado en equipo permitió desarrollar una aplicación organizada y funcional capaz de representar y almacenar estructuras fractales de manera eficiente.

REPORTE INFORMATIVO

Práctica Herencia sobreescritura

Concepto General

En este proyecto desarrollamos una aplicación en C++ basada en los principios de la Programación Orientada a Objetos, específicamente en los conceptos de herencia y sobreescritura de métodos. El sistema implementa una calculadora capaz de realizar operaciones matemáticas básicas y operaciones adicionales mediante una clase derivada que amplía las funcionalidades de la clase principal.

Además de realizar los cálculos, el programa almacena los resultados obtenidos en archivos TXT, CSV, XML y JSON, permitiendo conservar la información generada para futuras consultas.

Durante el desarrollo se aplicaron conceptos de herencia, reutilización de código, polimorfismo, validación de datos y manejo de archivos.

Definición

La herencia es un mecanismo de la Programación Orientada a Objetos que permite que una clase derivada herede atributos y métodos de una clase base.

La sobreescritura consiste en modificar o ampliar el comportamiento de métodos heredados para adaptarlos a nuevas necesidades.

En este proyecto se implementan dos clases:

Calculadora

Clase base encargada de realizar operaciones matemáticas básicas con una cantidad variable de números.

CalculadoraNueva

Clase derivada que amplía las capacidades de la calculadora incorporando nuevos métodos para realizar operaciones mediante algoritmos alternativos.

Objetivo

Desarrollar una aplicación que permita aplicar los conceptos de herencia y sobreescritura mediante la implementación de una calculadora con operaciones básicas y avanzadas.

- Aplicar los principios de herencia en C++.
- Reutilizar código mediante clases derivadas.
- Implementar operaciones matemáticas básicas.
- Desarrollar algoritmos alternativos para multiplicación y división.

- Calcular potencias mediante ciclos repetitivos.
- Generar archivos TXT, CSV, XML y JSON.
- Aplicar conceptos de programación orientada a objetos.

Proceso de elaboración

El proyecto fue desarrollado en pareja, distribuyendo las actividades de análisis, diseño, programación y pruebas para lograr una solución funcional y organizada.

1. Creación de funciones auxiliares

Inicialmente se implementaron funciones para procesar caracteres especiales durante la generación de archivos.

escaparJson()

Permite convertir caracteres especiales para mantener la sintaxis correcta de los archivos JSON.

escaparXml()

Convierte caracteres reservados de XML en entidades válidas.

escaparCsv()

Gestiona adecuadamente las comillas dentro de archivos CSV.

Estas funciones mejoran la compatibilidad de los archivos generados.

2. Implementación del sistema de almacenamiento

Se desarrolló la función `guardarArchivos()`, encargada de exportar la información generada por el programa en diferentes formatos.

Los datos almacenados incluyen:

- Nombre del programa.
- Números utilizados.
- Operación realizada.
- Resultado obtenido.

La información se guarda en:

- TXT
- CSV
- XML
- JSON

3. Desarrollo de la clase Calculadora

La clase Calculadora representa la clase base del proyecto.

Dentro de ella se almacenan:

- Los números ingresados por el usuario.
- El resultado obtenido.
- El tipo de operación ejecutada.

También se implementó la función leer(), encargada de capturar la cantidad de números que participarán en las operaciones.

Se incorporó una validación que limita la cantidad de números entre 1 y 100 para evitar errores durante la ejecución.

4. Implementación de la suma

Se desarrolló la función sum().

Esta operación recorre todos los números ingresados y acumula su valor para obtener el resultado final.

La operación queda registrada como:

suma_n_numeros

5. Implementación de la resta

La función res() utiliza el primer número como valor inicial y posteriormente resta cada uno de los números restantes.

El resultado se almacena para su posterior consulta y guardado.

6. Implementación de la multiplicación

La función mult() realiza la multiplicación convencional de todos los números ingresados.

Para ello se inicializa el resultado en uno y posteriormente se multiplican todos los elementos capturados.

7. Implementación de la división

La función div() realiza divisiones consecutivas utilizando el primer número como valor inicial.

Antes de cada división se verifica que el divisor sea diferente de cero para evitar errores matemáticos.

Si se detecta una división entre cero, el sistema muestra un mensaje de error y cancela la operación.

8. Desarrollo de la clase CalculadoraNueva

La clase CalculadoraNueva hereda las características de la clase Calculadora y agrega nuevas funcionalidades matemáticas.

Gracias a la herencia fue posible reutilizar atributos y métodos previamente desarrollados sin necesidad de duplicar código.

9. Multiplicación por sumas sucesivas

Se implementó una nueva versión de la multiplicación utilizando únicamente operaciones de suma.

El algoritmo consiste en sumar repetidamente un número tantas veces como indique el segundo valor.

Esta técnica permite comprender cómo se construye matemáticamente la multiplicación a partir de sumas repetidas.

10. División por restas sucesivas

Se desarrolló una versión alternativa de la división utilizando únicamente operaciones de resta.

El algoritmo resta repetidamente el divisor hasta que el resultado sea menor que este.

La cantidad de restas realizadas representa el cociente de la división.

También se incorpora una validación para evitar divisiones entre cero.

11. Cálculo de potencia

La función potencia() calcula la potencia de un número utilizando multiplicaciones repetitivas.

El algoritmo multiplica la base por sí misma tantas veces como indique el exponente.

Esta implementación permite comprender el funcionamiento interno de la operación de potenciación.

12. Conversión y almacenamiento de resultados

Se desarrolló la función numerosComoTexto(), encargada de convertir todos los números ingresados en una cadena de texto.

Posteriormente, la función obtenerDatos() organiza toda la información necesaria para ser exportada mediante los distintos formatos disponibles.

Finalmente, la función guardar() permite almacenar automáticamente los resultados obtenidos.

Comentario

Durante el desarrollo del proyecto se realizó una mejora en la generación de archivos XML agregando la declaración estándar al inicio del documento:

```
xml << "<?xml version=\"1.0\" encoding=\"UTF-8\" ?>\n";
```

Esta modificación permite que los documentos XML cumplan con los estándares internacionales de codificación y estructura, mejorando su compatibilidad con diferentes aplicaciones y sistemas.

Además, el proyecto permitió comprender de forma práctica cómo funciona la herencia entre clases y cómo una clase derivada puede ampliar las funcionalidades de una clase base mediante la incorporación de nuevos métodos especializados.

Conclusión

Con este proyecto logramos aplicar correctamente los conceptos de herencia y reutilización de código dentro de la Programación Orientada a Objetos. También reforzamos conocimientos sobre algoritmos matemáticos, validación de datos y generación de archivos en distintos formatos. El trabajo realizado en equipo permitió desarrollar una calculadora más completa y flexible, capaz de implementar tanto operaciones tradicionales como métodos alternativos basados en sumas y restas sucesivas.

REPORTE INFORMATIVO

Práctica Creación de dato

Concepto General

En este proyecto desarrollamos una aplicación en C++ que permite registrar información de automóviles y personas utilizando dos enfoques de programación diferentes: Programación Estructurada (PE) y Programación Orientada a Objetos (POO).

El programa permite capturar datos, mostrarlos en pantalla y almacenarlos en distintos formatos de archivo. De esta manera se pueden comparar las características de ambos paradigmas de programación y comprender cómo se organiza la información en cada uno de ellos.

Durante el desarrollo se aplicaron conceptos relacionados con estructuras, clases, encapsulamiento, manejo de datos, conversión de información y generación de archivos.

Definición

La Programación Estructurada es un paradigma que organiza la información mediante estructuras de datos y funciones independientes.

Por otro lado, la Programación Orientada a Objetos organiza la información mediante clases que agrupan atributos y métodos relacionados con una misma entidad.

En este proyecto se implementan ambos enfoques para representar dos tipos de datos:

- Automóviles.
- Personas.

Esto permite observar las diferencias entre trabajar con estructuras tradicionales y trabajar mediante objetos.

Objetivo

Desarrollar una aplicación que permita registrar y almacenar información de automóviles y personas utilizando tanto Programación Estructurada como Programación Orientada a Objetos.

- Aplicar conceptos de Programación Estructurada.
- Aplicar conceptos de Programación Orientada a Objetos.
- Capturar información de personas y automóviles.
- Mostrar los datos almacenados.
- Comparar ambos paradigmas de programación.
- Generar archivos TXT, CSV, XML y JSON.
- Reforzar conocimientos sobre organización y manejo de información.

Proceso de elaboración

El proyecto fue realizado en pareja, distribuyendo las actividades de análisis, diseño, programación y pruebas para lograr una implementación organizada y funcional.

1. Creación de funciones auxiliares

Inicialmente se desarrollaron funciones encargadas de procesar caracteres especiales para la generación de archivos.

`escaparJson()`

Permite convertir caracteres especiales para garantizar la correcta sintaxis de los archivos JSON.

`escaparXml()`

Convierte caracteres reservados de XML en entidades válidas para evitar errores de interpretación.

`escaparCsv()`

Gestiona adecuadamente el uso de comillas dentro de los archivos CSV.

Estas funciones mejoran la compatibilidad y estabilidad de los archivos generados.

2. Implementación del sistema de almacenamiento

Se desarrolló la función `guardarArchivos()`, encargada de exportar la información generada por el programa.

Los datos pueden almacenarse en los formatos:

- TXT
- CSV
- XML
- JSON

La información es organizada mediante pares de campo y valor para facilitar su lectura y procesamiento.

3. Desarrollo de la clase AutoPOO

Se implementó la clase `AutoPOO`, la cual representa un automóvil utilizando Programación Orientada a Objetos.

Esta clase almacena información relacionada con:

- Precio del automóvil.
- Año del automóvil.

La función leer() permite capturar los datos desde el teclado y la función mostrar() presenta la información en pantalla.

4. Obtención de datos del automóvil

Se desarrolló la función obtenerDatos() dentro de la clase AutoPOO.

Esta función organiza toda la información del objeto para que posteriormente pueda almacenarse en los diferentes formatos de archivo disponibles.

5. Desarrollo de la clase PersonaPOO

Se implementó la clase PersonaPOO, encargada de representar una persona mediante Programación Orientada a Objetos.

Los atributos almacenados son:

- Nombre.
- Apellido paterno.
- Apellido materno.
- Género.
- Edad.

La función leer() permite capturar la información del usuario y la función mostrar() muestra los datos organizados en pantalla.

6. Obtención de datos de la persona

La función obtenerDatos() recopila toda la información almacenada dentro del objeto y la prepara para su exportación.

Esta información incluye los datos personales capturados y la identificación del tipo de registro generado.

7. Implementación de Programación Estructurada

Además de las clases orientadas a objetos, el proyecto incorpora estructuras tradicionales para representar automóviles y personas.

Se desarrollaron las funciones:

`datosAutoPE()`

Obtiene la información de un automóvil representado mediante Programación Estructurada.

`datosPersonaPE()`

Obtiene la información de una persona representada mediante Programación Estructurada.

Estas funciones convierten la información almacenada en estructuras a un formato compatible con el sistema de almacenamiento implementado.

8. Comparación de paradigmas

El proyecto permite observar las diferencias entre ambos enfoques de programación.

Programación Estructurada

- Utiliza estructuras de datos simples.
- La información y las funciones se encuentran separadas.
- Su implementación es más sencilla para programas pequeños.

Programación Orientada a Objetos

- Utiliza clases y objetos.
- Agrupa datos y comportamientos dentro de una misma entidad.
- Facilita la reutilización y organización del código.

Esta comparación permite comprender las ventajas y aplicaciones de cada paradigma.

Comentario

Durante el desarrollo del proyecto se realizó una mejora en la generación de archivos XML agregando la declaración estándar al inicio del documento:

```
xml << "<?xml version=\"1.0\" encoding=\"UTF-8\" ?>\n";
```

Esta modificación permite que los documentos XML cumplan con los estándares internacionales de codificación y estructura, facilitando su interpretación por diferentes sistemas y aplicaciones.

Además, el proyecto permitió comparar de forma práctica dos paradigmas fundamentales de programación, identificando las diferencias en la organización de los datos y la manera en que cada enfoque gestiona la información.

Conclusión

Con este proyecto logramos aplicar conceptos de Programación Estructurada y Programación Orientada a Objetos dentro de una misma aplicación. También reforzamos conocimientos relacionados con estructuras, clases, encapsulamiento, manejo de datos y generación de archivos en distintos formatos. El trabajo realizado en equipo permitió desarrollar una solución organizada y funcional que facilita la comparación entre ambos paradigmas y demuestra la importancia de elegir la metodología adecuada según las necesidades del sistema.

REPORTE INFORMATIVO

Práctica Recursividad

Concepto General

En este proyecto desarrollamos una aplicación en C++ para implementar algoritmos recursivos que permiten calcular el factorial de un número y generar la serie de Fibonacci. La recursividad es una técnica de programación en la cual una función se llama a sí misma para resolver un problema dividiéndolo en versiones más pequeñas del mismo.

Además de realizar los cálculos, el programa almacena los resultados obtenidos en archivos TXT, CSV, XML y JSON, permitiendo conservar la información generada para futuras consultas.

Durante el desarrollo se aplicaron conceptos de funciones recursivas, casos base, validación de datos y manejo de archivos.

Definición

La recursividad es una técnica utilizada en programación donde una función puede llamarse a sí misma para resolver un problema. Cada llamada trabaja sobre una versión más pequeña del problema original hasta llegar a una condición de terminación conocida como caso base.

En este proyecto se implementan dos algoritmos recursivos:

- Cálculo del factorial.
- Generación de la serie de Fibonacci.

Ambos permiten comprender el funcionamiento y la aplicación práctica de la recursividad.

Objetivo

Desarrollar una aplicación que permita aplicar la técnica de recursividad para resolver problemas matemáticos como el factorial y la serie de Fibonacci.

- Aplicar funciones recursivas en C++.
- Comprender la importancia de los casos base.
- Calcular factoriales mediante recursividad.
- Generar términos de la serie Fibonacci.
- Validar datos ingresados por el usuario.
- Almacenar resultados en distintos formatos de archivo.
- Reforzar conocimientos sobre algoritmos recursivos.

Proceso de elaboración

El proyecto fue desarrollado en pareja, distribuyendo las actividades de diseño, programación, pruebas y documentación para lograr una solución funcional.

1. Creación de funciones auxiliares

Inicialmente se implementaron funciones encargadas de gestionar caracteres especiales para la generación de archivos.

`escaparJson()`

Permite adaptar caracteres especiales para que sean compatibles con el formato JSON.

`escaparXml()`

Convierte caracteres reservados en entidades válidas para XML.

`escaparCsv()`

Gestiona adecuadamente el uso de comillas dentro de archivos CSV.

Estas funciones garantizan la correcta generación de los archivos exportados.

2. Implementación del sistema de almacenamiento

Se desarrolló la función `guardarArchivos()`, encargada de exportar la información generada por el programa.

Los datos pueden almacenarse en:

- TXT
- CSV
- XML
- JSON

La información guardada incluye el número ingresado, la operación realizada y el resultado obtenido.

3. Desarrollo de la clase Recursividad

Se implementó la clase Recursividad, encargada de gestionar todas las operaciones del programa.

Dentro de esta clase se almacenan:

- El número ingresado por el usuario.
- La operación seleccionada.
- El resultado obtenido.

El constructor inicializa todas las variables necesarias para garantizar un funcionamiento correcto.

4. Captura y validación de datos

Se desarrolló la función leer() para solicitar al usuario un número entero.

La función incorpora una validación que obliga a ingresar únicamente valores mayores o iguales a cero.

Esta validación evita errores en los cálculos recursivos posteriores.

5. Implementación del factorial recursivo

Se desarrolló la función factorial(int x).

El algoritmo utiliza recursividad para calcular el factorial de un número.

Caso base

Cuando el valor es 0 o 1, la función devuelve 1.

Caso recursivo

La función multiplica el número actual por el factorial del número anterior.

De esta forma se construye el resultado hasta alcanzar el caso base.

6. Visualización del factorial

La función mostrarFactorial() ejecuta el cálculo del factorial utilizando el número ingresado.

Posteriormente:

- Almacena el resultado.
- Registra el tipo de operación realizada.
- Muestra el resultado en pantalla.

Esto permite conservar la información para su posterior almacenamiento.

7. Implementación de Fibonacci recursivo

Se desarrolló la función fibonacci(int x).

Esta función calcula cada término de la serie de Fibonacci mediante llamadas recursivas.

Casos base

- $\text{Fibonacci}(0) = 0$

- $\text{Fibonacci}(1) = 1$

Caso recursivo

Cada término se obtiene sumando los dos términos anteriores:

$$F(n) = F(n-1) + F(n-2)$$

8. Generación de la serie Fibonacci

La función `mostrarFibonacci()` genera una serie de términos de Fibonacci según el número indicado por el usuario.

Durante el proceso:

- Se calcula cada término mediante recursividad.
- Los resultados se muestran en pantalla.
- La secuencia completa se almacena en una cadena de texto.

Esto permite guardar posteriormente toda la serie generada.

9. Organización de los resultados

La función `obtenerDatos()` recopila toda la información generada durante la ejecución.

Entre los datos almacenados se encuentran:

- Nombre del programa.
- Número ingresado.
- Operación ejecutada.
- Resultado obtenido.

Estos datos son organizados para facilitar su exportación.

10. Exportación de la información

Finalmente, la función `guardar()` permite almacenar automáticamente los resultados obtenidos en los diferentes formatos de archivo disponibles.

De esta manera se conserva la información generada por los algoritmos recursivos.

Comentario

Durante el desarrollo del proyecto se realizó una mejora en la generación de archivos XML agregando la declaración estándar al inicio del documento:

```
xml << "<?xml version=\"1.0\" encoding=\"UTF-8\" ?>\n";
```

Esta modificación permite que los documentos XML cumplan con los estándares de codificación utilizados internacionalmente, mejorando su compatibilidad con diferentes programas y sistemas.

Además, este proyecto permitió comprender de forma práctica el funcionamiento de la recursividad, observando cómo una función puede resolver problemas complejos mediante llamadas a sí misma hasta alcanzar una condición de terminación.

Conclusión

Con este proyecto logramos aplicar correctamente la técnica de recursividad para resolver problemas matemáticos como la factorial y la serie de Fibonacci. También reforzamos conocimientos relacionados con validación de datos, manejo de funciones recursivas y generación de archivos en distintos formatos. El trabajo realizado en equipo permitió desarrollar una solución organizada y funcional que demuestra la utilidad de la recursividad para resolver problemas de manera elegante y eficiente.

REPORTE INFORMATIVO

Práctica Promedio

Concepto General

En este proyecto desarrollamos una aplicación en C++ que permite trabajar con un conjunto de cinco números ingresados por el usuario para realizar diferentes operaciones estadísticas básicas. Entre las operaciones implementadas se encuentran la suma de los valores, el cálculo del promedio, la identificación del número máximo y la identificación del número mínimo.

Además de mostrar los resultados en pantalla, el programa permite almacenar la información generada en archivos TXT, CSV, XML y JSON para facilitar su consulta posterior.

Durante el desarrollo se aplicaron conceptos relacionados con arreglos, procesamiento de datos, búsqueda de valores extremos, operaciones aritméticas y manejo de archivos.

Definición

El promedio es una medida estadística que se obtiene sumando un conjunto de valores y dividiendo el resultado entre la cantidad de elementos.

Por otra parte, los valores máximo y mínimo representan respectivamente el número más grande y el número más pequeño dentro de un conjunto de datos.

En este proyecto se implementan algoritmos que permiten realizar estas operaciones de forma automática a partir de cinco números proporcionados por el usuario.

Objetivo

Desarrollar una aplicación que permita realizar operaciones estadísticas básicas sobre un conjunto de cinco números y almacenar los resultados obtenidos en diferentes formatos de archivo.

- Capturar cinco valores numéricos.
- Calcular la suma de los números ingresados.
- Obtener el promedio de los datos.
- Determinar el valor máximo.
- Determinar el valor mínimo.
- Mostrar los resultados en pantalla.
- Exportar la información a archivos TXT, CSV, XML y JSON.

Proceso de elaboración

El proyecto fue desarrollado en pareja, distribuyendo las actividades de análisis, programación, pruebas y documentación para obtener una solución funcional y organizada.

1. Creación de funciones auxiliares

Inicialmente se implementaron funciones para gestionar caracteres especiales durante la generación de archivos.

escaparJson()

Permite adaptar caracteres especiales para que puedan almacenarse correctamente en archivos JSON.

escaparXml()

Convierte caracteres reservados de XML en entidades válidas para evitar errores de lectura.

escaparCsv()

Gestiona adecuadamente el uso de comillas dentro de archivos CSV.

Estas funciones garantizan la correcta estructura de los archivos exportados.

2. Implementación del sistema de almacenamiento

Se desarrolló la función `guardarArchivos()`, encargada de exportar los resultados obtenidos durante la ejecución del programa.

La información puede almacenarse en:

- TXT
- CSV
- XML
- JSON

Los archivos contienen información sobre los números capturados, la operación realizada y los resultados obtenidos.

3. Desarrollo de la clase Prom

Se implementó la clase `Prom`, encargada de administrar todas las operaciones matemáticas del programa.

Dentro de esta clase se almacenan:

- Los cinco números ingresados.
- La suma de los datos.
- El promedio calculado.
- El valor máximo.

- El valor mínimo.
- La operación realizada.
- El resultado obtenido.

El constructor inicializa todas las variables necesarias para garantizar un funcionamiento correcto.

4. Captura de datos

Se desarrolló la función leer(), encargada de solicitar al usuario cinco números.

Cada valor es almacenado dentro de un arreglo para su posterior procesamiento.

Esta información constituye la base para todas las operaciones implementadas.

5. Implementación de la suma

La función sum() recorre el arreglo de números y acumula cada valor para obtener la suma total.

Posteriormente:

- Se almacena el resultado.
- Se registra la operación realizada.
- Se muestra el resultado en pantalla.

Esta operación permite conocer el total de todos los números ingresados.

6. Implementación del promedio

Se desarrolló la función prom().

El algoritmo realiza los siguientes pasos:

1. Suma todos los números.
2. Divide la suma entre la cantidad de datos (5).
3. Almacena el resultado obtenido.

El promedio calculado representa el valor central de los datos ingresados.

7. Obtención del valor máximo y mínimo

La función maymen() permite identificar los valores extremos del conjunto de datos.

El proceso consiste en:

- Tomar inicialmente el primer número como referencia.
- Comparar cada elemento con el valor máximo actual.

- Comparar cada elemento con el valor mínimo actual.
- Actualizar los valores cuando sea necesario.

Al finalizar el recorrido se obtienen:

- El número más grande.
- El número más pequeño.

Ambos resultados se muestran en pantalla.

8. Conversión de números a texto

Se desarrolló la función `numerosComoTexto()`.

Su objetivo es convertir todos los números almacenados en una cadena de texto organizada.

Esta representación facilita la exportación de la información a los distintos formatos de archivo.

9. Organización de los datos

La función `obtenerDatos()` recopila toda la información generada durante la ejecución.

Entre los datos almacenados se encuentran:

- Nombre del programa.
- Números ingresados.
- Operación realizada.
- Resultado obtenido.

Cuando se ejecuta la búsqueda de máximo y mínimo, también se agregan ambos valores al registro final.

10. Exportación de resultados

Finalmente se desarrolló la función `guardar()`, encargada de almacenar toda la información procesada en los formatos disponibles.

De esta manera los resultados pueden consultarse posteriormente sin necesidad de volver a ejecutar los cálculos.

Comentario

Durante el desarrollo del proyecto se realizó una mejora en la generación de archivos XML agregando la declaración estándar al inicio del documento:

```
xml << "<?xml version=\"1.0\" encoding=\"UTF-8\" ?>\n";
```


Esta modificación permite que los documentos XML cumplan con los estándares de codificación internacional y puedan ser interpretados correctamente por diferentes sistemas y aplicaciones.

Además, este proyecto permitió comprender la importancia de las operaciones estadísticas básicas para el análisis de datos y reforzar conocimientos sobre arreglos, estructuras de control y almacenamiento de información.

Conclusión

Con este proyecto logramos desarrollar una aplicación capaz de realizar operaciones fundamentales de análisis numérico, tales como la suma, el promedio, la búsqueda del valor máximo y la búsqueda del valor mínimo. También fortalecimos conocimientos relacionados con arreglos, procesamiento de datos y generación de archivos en distintos formatos. El trabajo realizado en equipo permitió construir una solución organizada y funcional que facilita el análisis y almacenamiento de información numérica.

REPORTE INFORMATIVO

Práctica Calculadora

Concepto General

En este proyecto desarrollamos una calculadora en C++ capaz de realizar diversas operaciones matemáticas utilizando los principios de la Programación Orientada a Objetos. El programa permite trabajar con una cantidad variable de números ingresados por el usuario y realizar operaciones como suma, resta, multiplicación, división, promedio y búsqueda del número mayor.

Además, los resultados obtenidos pueden almacenarse en archivos TXT, CSV, XML y JSON para conservar la información generada durante la ejecución.

Durante el desarrollo se aplicaron conceptos de herencia, reutilización de código, validación de datos, manejo de arreglos y almacenamiento de información.

Definición

Una calculadora es una herramienta que permite realizar operaciones matemáticas sobre uno o varios valores numéricos.

En este proyecto se implementó una calculadora utilizando Programación Orientada a Objetos mediante una clase base y una clase derivada.

La clase principal se encarga de las operaciones matemáticas básicas, mientras que la clase hija amplía las funcionalidades incorporando nuevas operaciones como el cálculo del promedio y la búsqueda del número mayor.

Objetivo

Desarrollar una aplicación que permita realizar diversas operaciones matemáticas aplicando conceptos de herencia y reutilización de código en C++.

- Implementar una calculadora utilizando Programación Orientada a Objetos.
- Aplicar el concepto de herencia entre clases.
- Realizar operaciones matemáticas básicas.
- Calcular promedios.
- Determinar el número mayor de un conjunto de datos.
- Validar la información ingresada por el usuario.
- Exportar resultados en distintos formatos de archivo.

Proceso de elaboración

El proyecto fue desarrollado en pareja, distribuyendo las tareas de diseño, programación, pruebas y documentación para obtener una solución organizada y funcional.

1. Creación de funciones auxiliares

Inicialmente se desarrollaron funciones para procesar caracteres especiales durante la generación de archivos.

`escaparJson()`

Permite adaptar caracteres especiales para mantener una sintaxis válida en archivos JSON.

`escaparXml()`

Convierte caracteres reservados de XML en entidades válidas.

`escaparCsv()`

Gestiona correctamente el uso de comillas dentro de archivos CSV.

Estas funciones garantizan la compatibilidad de los archivos generados.

2. Implementación del sistema de almacenamiento

Se desarrolló la función `guardarArchivos()`, encargada de exportar la información generada por el programa.

Los resultados pueden almacenarse en:

- TXT
- CSV
- XML
- JSON

La información almacenada incluye los números utilizados, la operación realizada y el resultado obtenido.

3. Desarrollo de la clase Calculadora

Se implementó la clase `Calculadora`, considerada la clase base del proyecto.

Esta clase contiene:

- La cantidad de números a utilizar.
- Los valores ingresados.
- El resultado de la operación.
- El nombre de la operación ejecutada.

También incorpora los métodos necesarios para realizar las operaciones básicas.

4. Captura de datos

Se desarrolló la función leer() para solicitar al usuario la cantidad de números que participarán en las operaciones.

El sistema valida que la cantidad de datos se encuentre entre 1 y 100 elementos.

Posteriormente se capturan todos los números necesarios para realizar los cálculos.

5. Implementación de la suma

La función sum() recorre todos los números ingresados y acumula sus valores.

Al finalizar:

- Se almacena el resultado.
- Se registra la operación realizada.
- Se muestra el resultado en pantalla.

6. Implementación de la resta

La función res() toma el primer número como valor inicial y posteriormente resta cada uno de los números restantes.

El resultado final representa la diferencia obtenida después de realizar todas las operaciones de resta.

7. Implementación de la multiplicación

La función mult() calcula el producto de todos los números ingresados.

Para ello se inicializa el resultado en uno y posteriormente se multiplican todos los elementos almacenados.

8. Implementación de la división

La función div() realiza divisiones consecutivas utilizando el primer número como valor inicial.

Antes de realizar cada división se verifica que el divisor sea diferente de cero.

Si se detecta una división entre cero, el sistema muestra un mensaje de error y cancela la operación.

Esta validación evita errores matemáticos durante la ejecución.

9. Desarrollo de la clase CalculadoraHija

Se implementó la clase CalculadoraHija, la cual hereda las características de la clase Calculadora.

Gracias a la herencia fue posible reutilizar los atributos y métodos desarrollados en la clase principal.

La clase derivada incorpora nuevas funcionalidades sin modificar la estructura original del programa.

10. Implementación del promedio

La función prom() calcula el promedio de todos los números ingresados.

El procedimiento consiste en:

1. Sumar todos los valores.
2. Dividir la suma entre la cantidad de números capturados.

El resultado obtenido representa el valor promedio del conjunto de datos.

11. Implementación de la búsqueda del número mayor

La función may() permite identificar el valor más grande dentro del conjunto de números ingresados.

El algoritmo:

- Toma inicialmente el primer valor como referencia.
- Compara cada elemento con el valor mayor actual.
- Actualiza el valor cuando encuentra un número más grande.

Al finalizar el recorrido se obtiene el número mayor del conjunto.

12. Organización y almacenamiento de resultados

La función numerosComoTexto() convierte todos los números ingresados en una cadena de texto organizada.

Posteriormente, la función obtenerDatos() recopila:

- Nombre del programa.
- Números utilizados.
- Operación realizada.
- Resultado obtenido.

Finalmente, la función guardar() permite almacenar toda la información en los diferentes formatos disponibles.

Comentario

Durante el desarrollo del proyecto se realizó una mejora en la generación de archivos XML agregando la declaración estándar al inicio del documento:

```
xml << "<?xml version=\"1.0\" encoding=\"UTF-8\" ?>\n";
```

Esta modificación permite que los archivos XML cumplan con los estándares internacionales de codificación y puedan ser interpretados correctamente por diferentes aplicaciones.

Además, este proyecto permitió comprender de forma práctica cómo funciona la herencia en la Programación Orientada a Objetos, observando cómo una clase derivada puede ampliar las funcionalidades de una clase base sin duplicar código.

Conclusión

Con este proyecto logramos desarrollar una calculadora capaz de realizar múltiples operaciones matemáticas utilizando los principios de la Programación Orientada a Objetos. También reforzamos conocimientos relacionados con herencia, reutilización de código, validación de datos y generación de archivos en distintos formatos. El trabajo realizado en equipo permitió construir una solución organizada y eficiente que demuestra las ventajas de utilizar clases base y clases derivadas para ampliar funcionalidades dentro de una aplicación.

REPORTE INFORMATIVO

Práctica Calculadora básica

Concepto General

La calculadora básica es un programa desarrollado en C++ que permite realizar operaciones matemáticas fundamentales entre dos números. Su diseño se basa en la Programación Orientada a Objetos (POO), utilizando una clase que almacena los datos, ejecuta las operaciones y genera archivos con los resultados obtenidos.

Definición

Una calculadora básica es una herramienta informática que permite efectuar operaciones aritméticas elementales como suma, resta, multiplicación y división. En este proyecto se implementó mediante una clase llamada Calculadora, la cual recibe dos números, procesa la operación seleccionada por el usuario y muestra el resultado correspondiente.

Objetivo

Desarrollar un programa capaz de realizar operaciones matemáticas básicas entre dos valores numéricos, aplicando conceptos de programación orientada a objetos, validación de datos y generación de archivos en diferentes formatos para almacenar los resultados obtenidos.

Proceso de elaboración

Para la elaboración de este programa se realizaron las siguientes actividades:

- Se creó la clase Calculadora para encapsular los datos y métodos necesarios.
- Se definieron variables para almacenar los dos números ingresados por el usuario, el resultado y la operación seleccionada.
- Se implementó el método leer() para capturar los datos desde teclado.
- Se desarrolló el método operar() utilizando una estructura switch para ejecutar las operaciones matemáticas básicas.
- Se agregó una validación para evitar divisiones entre cero.
- Se implementó el método mostrar() para presentar el resultado en pantalla.
- Se creó el método obtenerDatos() para organizar la información generada.
- Se desarrolló la función guardarArchivos() para exportar los resultados en formatos TXT, CSV, XML y JSON.
- Se añadieron funciones de escape para garantizar que los caracteres especiales se almacenen correctamente en XML, JSON y CSV.

Comentario

Durante el desarrollo del programa se incorporó la generación automática de archivos en formatos TXT, CSV, XML y JSON, permitiendo conservar los resultados de las operaciones realizadas.

Además, se agregó la siguiente línea al archivo XML para incluir la declaración estándar del documento:

```
xml << "<?xml version=\"1.0\" encoding=\"UTF-8\" ?>\n";
```

Esta modificación permite que el archivo XML cumpla con la estructura recomendada para su correcta interpretación por diferentes aplicaciones y sistemas.

Conclusión

Con la realización de este programa logramos aplicar los conocimientos básicos de programación orientada a objetos en C++, implementando una calculadora capaz de efectuar operaciones aritméticas fundamentales de manera sencilla y eficiente. Además, aprendimos a validar datos para evitar errores durante la ejecución, como la división entre cero.

También reforzamos el manejo de archivos al almacenar la información generada en distintos formatos (TXT, CSV, XML y JSON), lo que permite una mejor organización y conservación de los resultados. En general, esta práctica nos ayudó a comprender la importancia de estructurar correctamente un programa y de utilizar herramientas que faciliten el procesamiento y almacenamiento de datos.

REPORTE INFORMATIVO

Práctica Suma dos números

Concepto General

La suma es una de las operaciones matemáticas básicas más utilizadas en programación. Este programa fue desarrollado en C++ para solicitar dos números al usuario, realizar la operación de suma y almacenar los resultados en diferentes formatos de archivo mediante el uso de programación orientada a objetos.

Definición

El programa "Suma de Dos Números" consiste en una aplicación que permite capturar dos valores numéricos, calcular su suma y mostrar el resultado obtenido. Además, genera archivos de salida en formatos TXT, CSV, XML y JSON para conservar la información procesada.

Objetivo

Desarrollar un programa que permita realizar la suma de dos números ingresados por el usuario, aplicando conceptos básicos de programación orientada a objetos, manejo de funciones y almacenamiento de información en distintos formatos de archivo.

Proceso de elaboración

Para desarrollar este programa se llevaron a cabo las siguientes actividades:

- Se creó la clase Sum2 para encapsular los datos y operaciones del programa.
- Se declararon las variables necesarias para almacenar los dos números y el resultado de la suma.
- Se implementó el método leer() para capturar los datos proporcionados por el usuario.
- Se desarrolló el método sum() encargado de realizar la operación matemática y mostrar el resultado.
- Se creó el método obtenerDatos() para organizar la información generada por el programa.
- Se implementó la función guardarArchivos() para exportar los datos en formatos TXT, CSV, XML y JSON.
- Se añadieron funciones de escape para garantizar la correcta escritura de caracteres especiales en los archivos generados.

Comentario

Durante el desarrollo del programa se incorporó la funcionalidad de almacenamiento de datos en múltiples formatos, permitiendo conservar la información de entrada y el resultado de la operación realizada.

Asimismo, se agregó la siguiente línea al archivo XML para incluir la declaración estándar del documento:

```
xml << "<?xml version=\"1.0\" encoding=\"UTF-8\" ?>\n";
```

Esta mejora permite que el archivo XML tenga una estructura más completa y compatible con diferentes sistemas y aplicaciones.

Conclusión

Con la elaboración de este programa reforzamos los conocimientos básicos sobre programación orientada a objetos y operaciones matemáticas en C++. También aprendimos a capturar datos desde teclado, procesarlos mediante funciones y almacenar los resultados en distintos formatos de archivo.

Además, comprendimos la importancia de organizar adecuadamente la información generada por un programa para facilitar su consulta y reutilización. Esta práctica sirvió como base para desarrollar programas más complejos que requieran procesamiento y almacenamiento de datos.

REPORTE INFORMATIVO

Práctica Hola mundo

Concepto General

El programa "Hola Mundo" es una de las prácticas más básicas en programación. Su finalidad es demostrar el funcionamiento de la entrada y salida de datos, permitiendo al usuario ingresar un mensaje para posteriormente mostrarlo en pantalla y almacenarlo en distintos formatos de archivo.

Definición

Se trata de un programa desarrollado en C++ que permite capturar una cadena de texto ingresada por el usuario, mostrarla en pantalla y guardar la información en archivos TXT, CSV, XML y JSON mediante el uso de programación orientada a objetos y manejo de archivos.

Objetivo

Desarrollar una aplicación sencilla que permita capturar y mostrar mensajes de texto, reforzando el uso de cadenas de caracteres, entrada y salida de datos, programación orientada a objetos y almacenamiento de información en diferentes formatos.

Proceso de elaboración

Para la elaboración de este programa se realizaron las siguientes actividades:

- Se creó la clase Holamundo para administrar el mensaje ingresado por el usuario.
- Se declaró una variable tipo cadena para almacenar el texto capturado.
- Se implementó el método leer() para recibir el mensaje mediante teclado.
- Se desarrolló el método mos() para mostrar el contenido almacenado en pantalla.
- Se creó el método obtenerDatos() para organizar la información generada.
- Se implementó la función guardarArchivos() para exportar los datos en formatos TXT, CSV, XML y JSON.
- Se añadieron funciones de escape para garantizar el correcto almacenamiento de caracteres especiales en los distintos formatos de archivo.

Comentario

Durante el desarrollo del programa se incorporó la funcionalidad de guardar el mensaje ingresado por el usuario en varios formatos de almacenamiento, permitiendo conservar la información para futuras consultas.

Además, se agregó la siguiente línea al archivo XML para incluir la declaración estándar del documento:

```
xml << "<?xml version=\"1.0\" encoding=\"UTF-8\" ?>\n";
```

Esta modificación mejora la estructura del archivo XML y facilita su correcta interpretación por diferentes aplicaciones y sistemas.

Conclusión

Con la realización de este programa reforzamos los conocimientos básicos de programación en C++, especialmente en el manejo de cadenas de texto, entrada y salida de datos y programación orientada a objetos. Asimismo, aprendimos a almacenar información en distintos formatos de archivo, permitiendo una mejor organización y conservación de los datos.

Esta práctica sirvió como una introducción al desarrollo de aplicaciones más complejas, ya que permitió comprender conceptos fundamentales que serán utilizados en proyectos de mayor tamaño y funcionalidad.

REPORTE INFORMATIVO

Práctica Quick Sort

Concepto General

En este proyecto desarrollamos una aplicación en C++ que permite leer datos numéricos desde un archivo de texto, ordenarlos mediante el algoritmo QuickSort y posteriormente almacenarlos en distintos formatos de archivo. El programa fue diseñado para controlar que las acciones se realicen en una secuencia específica: primero leer el archivo, después ordenar los datos y finalmente guardarlos antes de salir del sistema.

El proyecto combina conceptos de estructuras de datos, algoritmos de ordenamiento, manejo dinámico de memoria y procesamiento de archivos, permitiendo poner en práctica conocimientos fundamentales de programación orientada a objetos.

Definición

QuickSort es un algoritmo de ordenamiento basado en la estrategia de "divide y vencerás". Su funcionamiento consiste en seleccionar un elemento denominado pivote y reorganizar los demás elementos de tal forma que los menores queden a su izquierda y los mayores a su derecha. Posteriormente, el proceso se repite de manera recursiva en cada una de las particiones hasta que todo el arreglo queda ordenado.

Este algoritmo es ampliamente utilizado debido a su eficiencia y rapidez para ordenar grandes cantidades de datos.

Objetivo

Desarrollar un programa en C++ capaz de leer datos desde un archivo de texto, ordenarlos utilizando el algoritmo QuickSort y exportar los resultados en diferentes formatos de almacenamiento.

- Implementar el algoritmo QuickSort para el ordenamiento de datos.
- Leer información numérica desde archivos de texto.
- Utilizar memoria dinámica para almacenar los datos.
- Validar la secuencia correcta de ejecución del programa.
- Exportar la información ordenada en formatos TXT, CSV, XML y JSON.
- Aplicar conceptos de programación orientada a objetos.

Proceso de Elaboración

Al desarrollar este proyecto trabajamos en equipo para dividir las actividades y facilitar la implementación del sistema.

1. Diseño de la clase Quick

Inicialmente se creó la clase Quick, la cual concentra todas las funciones necesarias para el manejo de archivos y el ordenamiento de datos. También se definieron variables para controlar el estado del programa:

- archivoLeido

- archivoOrdenado
- archivoGrabado

Estas variables permiten verificar que el usuario siga la secuencia correcta de ejecución.

2. Implementación del algoritmo QuickSort

Posteriormente se desarrolló el algoritmo QuickSort mediante dos funciones principales:

Función particion()

Esta función selecciona el último elemento del arreglo como pivote y reorganiza los elementos menores y mayores alrededor de él.

Función quickSort()

Se encarga de llamar recursivamente a la función de partición hasta ordenar completamente el arreglo.

3. Lectura del archivo

Se implementó la función leerArchivo() para:

- Solicitar el nombre del archivo.
- Abrir el archivo de texto.
- Contar la cantidad de números existentes.
- Reservar memoria dinámica.
- Cargar todos los datos en el arreglo.

Además, se incluyeron validaciones para evitar leer archivos inexistentes o vacíos.

4. Visualización de datos

Se creó la función mostrarArreglo() para visualizar en pantalla el contenido almacenado en memoria y verificar que los datos fueron cargados correctamente.

5. Ordenamiento de datos

Mediante la función ordenarQuickSort() se ejecuta el algoritmo QuickSort únicamente cuando existe un archivo previamente cargado.

También se evita ordenar varias veces el mismo conjunto de datos mediante una validación de estado.

6. Generación de archivos

Una vez ordenados los datos, se implementó la función guardarArchivo() para exportar la información en cuatro formatos diferentes:

TXT

Almacena un número por línea.

CSV

Genera un archivo compatible con hojas de cálculo.

XML

Organiza la información mediante etiquetas estructuradas.

JSON

Guarda los datos en formato de arreglo para aplicaciones web y sistemas modernos.

7. Validación de salida

La función salir() verifica que el usuario haya completado todas las etapas del proceso antes de cerrar el programa:

1. Leer archivo.
2. Ordenar archivo.
3. Guardar archivo.

Si alguna etapa no se ha realizado, el sistema impide la salida.

Funcionamiento General del Programa

1. El usuario ingresa el nombre del archivo.
2. El programa lee los números almacenados.
3. Los datos son cargados en memoria dinámica.
4. Se muestran los números originales.
5. Se ejecuta QuickSort para ordenarlos.
6. Los resultados se guardan en TXT, CSV, XML y JSON.
7. El sistema permite salir únicamente después de completar todo el proceso.

Comentario

Durante el desarrollo del proyecto se realizó una mejora en la generación del archivo XML agregando la declaración estándar del documento al inicio del archivo:

```
xml << "<?xml version=\"1.0\" encoding=\"UTF-8\" ?>\n";
```

Esta modificación permite que el archivo XML cumpla con los estándares de estructura y codificación utilizados por la mayoría de los sistemas y aplicaciones que procesan documentos XML.

Gracias a esta mejora, los archivos generados presentan una mayor compatibilidad y pueden ser interpretados correctamente por diferentes plataformas y herramientas de análisis de datos.

Conclusión

Con este proyecto logramos implementar correctamente el algoritmo QuickSort en C++, integrándolo con el manejo de archivos y la exportación de información en distintos formatos. Además de reforzar conceptos de ordenamiento y programación orientada a objetos, aprendimos la importancia de validar procesos, administrar memoria dinámica y generar archivos estructurados que puedan utilizarse en diferentes aplicaciones. El trabajo en equipo permitió distribuir responsabilidades y desarrollar una solución más organizada y eficiente.

REPORTE INFORMATIVO

Práctica Pila puntero

Concepto General

Una pila es una estructura de datos lineal que trabaja bajo el principio LIFO (Last In, First Out), lo que significa que el último elemento que entra es el primero en salir. En este programa se implementó una pila dinámica utilizando punteros y nodos enlazados para almacenar valores enteros de forma eficiente.

Definición

El programa "Pila Dinámica con Punteros" permite insertar, eliminar y mostrar elementos dentro de una pila utilizando memoria dinámica. Además, verifica que no existan elementos repetidos y permite guardar la información almacenada en archivos TXT, CSV, XML y JSON.

Objetivo

Desarrollar una estructura de datos tipo pila utilizando listas enlazadas y punteros, aplicando conceptos de memoria dinámica, manejo de nodos y almacenamiento de información en distintos formatos de archivo.

Proceso de Elaboración

Para la elaboración de este programa se realizaron las siguientes actividades:

- Se creó una estructura de nodo para almacenar cada valor y el enlace al siguiente elemento.
- Se declaró un puntero llamado tope para señalar el elemento superior de la pila.
- Se implementó el método push() para insertar nuevos elementos en la pila.
- Se agregó una validación para evitar que existan valores repetidos.
- Se desarrolló el método pop() para eliminar el elemento ubicado en la parte superior de la pila.
- Se implementó el método mos() para mostrar todos los elementos almacenados.
- Se creó el método vacia() para verificar si la pila contiene elementos.
- Se desarrolló el método llena() para comprobar la disponibilidad de memoria dinámica.
- Se implementó el método tam() para calcular la cantidad de elementos almacenados.
- Se creó el método guardarArchivo () para exportar los datos a archivos TXT, CSV, XML y JSON.

Comentario

Durante el desarrollo del programa se utilizó memoria dinámica para administrar los nodos de la pila, permitiendo agregar y eliminar elementos sin necesidad de definir un tamaño fijo previamente.

Asimismo, se agregó la siguiente línea al archivo XML para incluir la declaración estándar del documento:

```
xml << "<?xml version=\"1.0\" encoding=\"UTF-8\" ?>\n";
```

Esta mejora permite que el archivo XML tenga una estructura más completa y compatible con diferentes aplicaciones encargadas de procesar este tipo de documentos.

Conclusión

Con la realización de este programa logramos comprender el funcionamiento de las estructuras de datos dinámicas mediante el uso de punteros y memoria dinámica. También reforzamos conceptos relacionados con listas enlazadas, inserción y eliminación de elementos bajo el principio LIFO.

Además, aprendimos la importancia de validar la información antes de almacenarla y de generar archivos en distintos formatos para conservar los datos de manera organizada. Esta práctica fortaleció nuestros conocimientos sobre estructuras dinámicas y su aplicación en el desarrollo de software.

REPORTE INFORMATIVO

Práctica Merge sort

Concepto General

Merge Sort es un algoritmo de ordenamiento que organiza una colección de datos siguiendo la estrategia "divide y vencerás". Consiste en dividir repetidamente un conjunto de elementos en partes más pequeñas, ordenar cada una de ellas y posteriormente unir las de forma ordenada hasta obtener el conjunto completo ordenado.

Definición

El programa implementa el algoritmo Merge Sort para ordenar números enteros almacenados en un archivo de texto. Los datos se leen desde un archivo, se guardan en un arreglo dinámico y posteriormente se ordenan mediante llamadas recursivas. Finalmente, los resultados pueden guardarse en archivos TXT, CSV, XML y JSON.

Objetivo

Desarrollar una aplicación que permita leer números desde un archivo, ordenarlos utilizando el algoritmo Merge Sort y almacenar los resultados en distintos formatos de archivo, aplicando conceptos de recursividad, manejo dinámico de memoria y manipulación de archivos en C++.

Proceso de Elaboración

1. Se creó una clase encargada de administrar la lectura, ordenamiento y almacenamiento de datos.
2. Se implementó la lectura de números desde un archivo de texto utilizando flujos de entrada.
3. Se reservó memoria dinámicamente para almacenar los datos leídos.
4. Se desarrolló la función `mergeSort()`, encargada de dividir el arreglo en subarreglos cada vez más pequeños.
5. Se implementó la función `merge()`, responsable de combinar los subarreglos manteniendo el orden correcto de los elementos.
6. Se agregó la visualización del arreglo antes y después del ordenamiento.
7. Se implementó la exportación de los resultados en formatos TXT, CSV, XML y JSON.
8. Se incorporaron validaciones para asegurar que el archivo fuera leído y guardado correctamente antes de finalizar el programa.

Comentario

La elaboración de este programa permitió comprender de manera práctica el funcionamiento de uno de los algoritmos de ordenamiento más eficientes. También ayudó a reforzar conocimientos

relacionados con recursividad, manejo de memoria dinámica, lectura y escritura de archivos, así como la organización de información en distintos formatos de almacenamiento.

Conclusión

Este proyecto permitió aplicar conceptos fundamentales de programación orientada a objetos y algoritmos de ordenamiento mediante la implementación de Merge Sort. Gracias al uso de recursividad y la técnica de división y combinación de datos, fue posible ordenar eficientemente los números obtenidos de un archivo. Además, la generación de archivos TXT, CSV, XML y JSON demuestra la importancia de la gestión y almacenamiento estructurado de información, fortaleciendo habilidades esenciales para el desarrollo de software.

REPORTE INFORMATIVO

Práctica Lista puntero

Concepto General

Una lista enlazada es una estructura de datos dinámica formada por nodos conectados mediante punteros. Cada nodo almacena un dato y una referencia al siguiente nodo de la lista, permitiendo agregar o eliminar elementos sin necesidad de mover los demás datos en memoria.

Definición

El programa implementa una lista enlazada simple utilizando punteros en C++. Permite insertar valores sin duplicados, eliminar elementos específicos, mostrar el contenido de la lista, verificar si está vacía o llena, obtener su tamaño y guardar los datos en archivos TXT, CSV, XML y JSON.

Objetivo

Desarrollar una aplicación que utilice listas enlazadas para almacenar y administrar datos dinámicamente mediante punteros, aplicando operaciones básicas de inserción, eliminación, recorrido y almacenamiento de información en diferentes formatos de archivo.

Proceso de Elaboración

1. Se definió una estructura Nodo que contiene un dato entero y un puntero al siguiente nodo.
2. Se creó una clase encargada de administrar la lista enlazada mediante un puntero llamado cabeza.
3. Se implementó la función de inserción para agregar nuevos elementos al final de la lista, evitando valores repetidos.
4. Se desarrolló la función de eliminación para remover elementos específicos de la lista.
5. Se creó una función para mostrar todos los elementos almacenados recorriendo la lista nodo por nodo.
6. Se implementaron métodos para verificar si la lista está vacía o si existe suficiente memoria disponible para seguir agregando elementos.
7. Se añadió una función para calcular el tamaño de la lista contando la cantidad de nodos existentes.
8. Se desarrolló un sistema de exportación de datos que guarda la información en formatos TXT, CSV, XML y JSON.
9. Se incorporaron validaciones para evitar errores durante las operaciones de inserción, eliminación y almacenamiento.

Comentario

Este programa permitió comprender el funcionamiento de las estructuras dinámicas de datos mediante el uso de punteros. A diferencia de los arreglos, las listas enlazadas ofrecen mayor flexibilidad para insertar y eliminar elementos, ya que no requieren reorganizar toda la información almacenada. Además, el manejo de archivos complementó el aprendizaje sobre persistencia de datos.

Conclusión

La implementación de una lista enlazada con punteros permitió aplicar conceptos fundamentales de estructuras de datos dinámicas y gestión de memoria en C++. A través de las operaciones de inserción, eliminación y recorrido se observó cómo los nodos pueden enlazarse eficientemente mediante direcciones de memoria. Asimismo, la posibilidad de guardar la información en distintos formatos fortaleció el conocimiento sobre manejo de archivos y organización de datos, obteniendo una solución flexible para almacenar y administrar información.

REPORTE INFORMATIVO

Práctica Cola puntero

Concepto General

Una cola es una estructura de datos lineal que funciona siguiendo el principio FIFO (First In, First Out), es decir, el primer elemento que entra es el primero que sale. Cuando se implementa mediante punteros, los elementos se almacenan dinámicamente en nodos enlazados, permitiendo un uso flexible de la memoria.

Definición

El programa implementa una cola dinámica utilizando punteros en C++. Permite insertar elementos al final de la cola, eliminar elementos desde el frente, mostrar el contenido almacenado, obtener el tamaño de la estructura y guardar los datos en archivos TXT, CSV, XML y JSON. Además, evita el ingreso de valores duplicados.

Objetivo

Desarrollar una aplicación que permita administrar datos mediante una cola enlazada, aplicando conceptos de estructuras dinámicas, manejo de memoria con punteros y almacenamiento de información en diferentes formatos de archivo.

Proceso de Elaboración

1. Se definió una estructura Nodo que almacena un dato y un puntero al siguiente nodo.
2. Se creó una clase encargada de administrar la cola mediante dos punteros: frente y final.
3. Se implementó la operación de inserción (enqueue) para agregar elementos al final de la cola.
4. Se añadió una validación para impedir que existan elementos repetidos dentro de la estructura.
5. Se desarrolló la operación de eliminación (dequeue), que retira el elemento ubicado al frente de la cola.
6. Se implementó una función para mostrar todos los elementos almacenados recorriendo los nodos enlazados.
7. Se creó una función para verificar si la cola se encuentra vacía.
8. Se implementó un contador que mantiene actualizado el tamaño de la cola.
9. Se desarrolló un sistema de exportación de datos para guardar la información en archivos TXT, CSV, XML y JSON.
10. Se incorporaron validaciones para evitar operaciones inválidas cuando la cola está vacía.

Comentario

La realización de este programa permitió comprender el funcionamiento de las colas dinámicas y la forma en que los punteros facilitan la administración eficiente de memoria. También ayudó a reforzar conocimientos sobre estructuras de datos lineales, operaciones FIFO y manipulación de archivos para almacenar información de manera organizada.

Conclusión

Este proyecto permitió aplicar los conceptos fundamentales de las colas implementadas mediante punteros, observando cómo los elementos se administran siguiendo el principio FIFO. El uso de memoria dinámica hizo posible agregar y eliminar datos de forma eficiente sin depender de un tamaño fijo. Además, la capacidad de exportar la información en distintos formatos fortaleció las habilidades relacionadas con el manejo de archivos y la persistencia de datos, obteniendo una solución práctica para la gestión ordenada de información.

REPORTE INFORMATIVO

Práctica Pila puntero

Concepto General

El método Burbuja (Bubble Sort) es un algoritmo de ordenamiento que compara pares de elementos adyacentes y los intercambia cuando se encuentran en el orden incorrecto. Este proceso se repite varias veces hasta que todos los elementos quedan ordenados.

Definición

El programa implementa el algoritmo de ordenamiento Burbuja para organizar números enteros almacenados en un archivo de texto. Los datos son leídos desde el archivo, almacenados en un arreglo dinámico y ordenados mediante comparaciones sucesivas. Posteriormente, los resultados pueden guardarse en formatos TXT, CSV, XML y JSON.

Objetivo

Desarrollar una aplicación que permita leer números desde un archivo, ordenarlos utilizando el método Burbuja y almacenar los resultados en distintos formatos de archivo, aplicando conceptos de arreglos dinámicos, algoritmos de ordenamiento y manejo de archivos en C++.

Proceso de Elaboración

1. Se creó una clase encargada de administrar la lectura, ordenamiento y almacenamiento de los datos.
2. Se implementó la lectura de números desde un archivo de texto utilizando flujos de entrada.
3. Se utilizó memoria dinámica para almacenar los datos leídos en un arreglo.
4. Se desarrolló el algoritmo Burbuja mediante ciclos anidados que comparan elementos consecutivos.
5. Se realizaron intercambios entre elementos cuando estos se encontraban en un orden incorrecto.
6. Se agregó una función para mostrar el contenido del arreglo antes y después del ordenamiento.
7. Se implementó la exportación de los datos ordenados en archivos TXT, CSV, XML y JSON.
8. Se incorporaron validaciones para asegurar que el archivo haya sido leído y ordenado antes de guardarlo.
9. Se añadieron controles para evitar operaciones repetidas o incorrectas durante la ejecución del programa.

Comentario

Este programa permitió comprender el funcionamiento de uno de los algoritmos de ordenamiento más conocidos y sencillos de implementar. Aunque no es el más eficiente para grandes cantidades de datos, resulta ideal para comprender la lógica de comparación e intercambio de elementos. Además, fortaleció conocimientos relacionados con archivos, arreglos dinámicos y estructuras básicas de programación.

Conclusión

La implementación del método Burbuja permitió aplicar conceptos fundamentales de ordenamiento de datos mediante comparaciones sucesivas entre elementos. A través del uso de arreglos dinámicos y manejo de archivos, fue posible organizar información numérica de manera automática y almacenarla en diferentes formatos. Este proyecto ayudó a reforzar conocimientos de algoritmos, estructuras de datos y persistencia de información, constituyendo una base importante para el estudio de técnicas de ordenamiento más avanzadas.

REPORTE INFORMATIVO

Práctica Grafo

Concepto General

Un grafo es una estructura de datos formada por nodos (vértices) y conexiones (aristas) que permiten representar relaciones entre diferentes elementos. Los grafos son ampliamente utilizados para modelar redes de transporte, rutas, mapas, redes sociales y sistemas de comunicación.

Definición

Este programa implementa un grafo en C++ mediante nodos que representan lugares y conexiones que representan rutas entre ellos. Cada conexión almacena información como tiempo de recorrido y costo. Además, incorpora el algoritmo de Dijkstra para encontrar las rutas de menor costo entre un nodo de origen y los demás nodos del grafo.

Objetivo

Desarrollar una aplicación capaz de crear, almacenar, visualizar y analizar grafos, permitiendo cargar información desde archivos XML y JSON, calcular rutas óptimas mediante el algoritmo de Dijkstra y exportar los resultados a diferentes formatos de archivo.

Proceso de Elaboración

1. Se definen las estructuras necesarias para representar nodos, conexiones y rutas del grafo.
2. Se implementan funciones para agregar nodos y conexiones entre ellos.
3. Cada conexión almacena información relevante como nombre de la ruta, tiempo y costo.
4. Se desarrolla una función para mostrar toda la información del grafo en pantalla.
5. Se implementa el algoritmo de Dijkstra para calcular el camino de menor costo desde un nodo de origen hacia todos los demás nodos.
6. Se crean funciones para leer la información del grafo desde archivos XML y JSON.
7. Los datos cargados son almacenados en memoria mediante vectores dinámicos.
8. Se generan archivos de salida en formatos TXT, CSV, XML y JSON que contienen toda la información del grafo y sus conexiones.
9. El sistema permite visualizar rutas, nodos, costos y tiempos de recorrido de forma organizada.

Comentario

Este proyecto integra conceptos fundamentales de estructuras de datos, manejo de archivos y algoritmos de optimización. La implementación del algoritmo de Dijkstra permite resolver problemas reales relacionados con búsqueda de rutas mínimas, mientras que el soporte para XML y JSON facilita el intercambio de información entre diferentes sistemas.

Conclusión

La implementación de grafos permite modelar y analizar redes complejas de manera eficiente. Gracias al uso del algoritmo de Dijkstra, es posible encontrar rutas óptimas considerando los costos asociados a cada conexión. Además, la capacidad de importar y exportar información en múltiples formatos fortalece la interoperabilidad del sistema y demuestra la aplicación práctica de estructuras de datos avanzadas en problemas del mundo real.

REPORTE INFORMATIVO

Práctica Dígrafo

Concepto General

Un dígrafo (grafo dirigido) es una estructura de datos compuesta por nodos y conexiones orientadas llamadas aristas. A diferencia de un grafo simple, las conexiones tienen una dirección específica, permitiendo representar rutas, flujos o relaciones donde el sentido del recorrido es importante.

Definición

El programa implementa un dígrafo utilizando programación orientada a objetos en C++. Permite agregar nodos y conexiones dirigidas entre ellos, almacenar información asociada a cada ruta (nombre de la conexión, tiempo y costo), visualizar la estructura completa, calcular rutas mínimas mediante el algoritmo de Dijkstra, leer información desde archivos XML y JSON, y generar archivos de salida en formatos TXT, CSV, XML y JSON.

Objetivo

Desarrollar una aplicación capaz de modelar y administrar un dígrafo dirigido, permitiendo representar rutas entre diferentes lugares, calcular caminos de menor costo mediante el algoritmo de Dijkstra y almacenar la información en distintos formatos para su posterior consulta y análisis.

Proceso de Elaboración

1. Se definieron las estructuras necesarias para representar nodos, conexiones y rutas dentro del dígrafo.
2. Se implementó la función para agregar nodos identificados mediante un ID y una etiqueta descriptiva.
3. Se desarrolló el método para agregar conexiones dirigidas entre nodos, incluyendo información de tiempo, costo y nombre de la arista.
4. Se creó una función para mostrar en pantalla toda la información del dígrafo, incluyendo nodos, aristas y rutas.
5. Se implementó el algoritmo de Dijkstra para encontrar las rutas de menor costo desde un nodo origen hacia los demás nodos del sistema.
6. Se añadieron funciones para importar información desde archivos XML y JSON.
7. Se desarrollaron métodos para exportar toda la información del dígrafo en archivos TXT, CSV, XML y JSON.
8. Finalmente, se realizaron pruebas para verificar el correcto almacenamiento, lectura y cálculo de rutas dentro del sistema.

Comentario

Este programa permitió aplicar conceptos avanzados de estructuras de datos y teoría de grafos. La implementación de un dígrafo combinado con el algoritmo de Dijkstra demuestra cómo pueden resolverse problemas reales relacionados con rutas, transporte, logística y redes de comunicación. Además, la capacidad de importar y exportar información en distintos formatos facilita el intercambio y la persistencia de los datos.

Conclusión

La implementación del dígrafo permitió comprender el funcionamiento de los grafos dirigidos y su aplicación en la representación de rutas entre diferentes ubicaciones. El uso del algoritmo de Dijkstra proporcionó una solución eficiente para determinar caminos de menor costo, mientras que las funciones de lectura y escritura de archivos permitieron gestionar la información de forma organizada. En conjunto, el proyecto integra estructuras de datos, algoritmos de optimización y manejo de archivos, fortaleciendo habilidades fundamentales en el desarrollo de software.

REPORTE INFORMATIVO

Práctica Árbol B

Concepto General

Un Árbol Binario de Búsqueda (Binary Search Tree o BST) es una estructura de datos jerárquica compuesta por nodos conectados entre sí. Cada nodo contiene un dato y puede tener hasta dos hijos: uno izquierdo y uno derecho. La característica principal es que los valores menores se almacenan a la izquierda y los mayores a la derecha, lo que facilita las operaciones de búsqueda, inserción y recorrido.

Definición

El programa implementa un Árbol Binario de Búsqueda utilizando estructuras dinámicas y punteros. Los nodos representan lugares identificados mediante un ID y una etiqueta. Además, incorpora conexiones entre nodos mediante rutas que almacenan información como tiempo y costo, permitiendo aplicar el algoritmo de Dijkstra para encontrar caminos mínimos.

Objetivo

Desarrollar una estructura de árbol binario de búsqueda capaz de:

- Almacenar nodos organizados por identificador.
- Gestionar conexiones entre nodos.
- Realizar recorridos Inorden, Preorden y Posorden.
- Calcular rutas de costo mínimo utilizando Dijkstra.
- Leer información desde archivos XML y JSON.
- Exportar resultados en formatos TXT, CSV, XML y JSON.

Proceso de Elaboración

1. Se define la estructura de datos para representar nodos, conexiones y rutas.
2. Se implementa la función `agregarNodo()` para insertar elementos dentro del árbol respetando las reglas del BST.
3. Se desarrolla la función `agregarConexion()` para registrar relaciones entre nodos con información de tiempo y costo.
4. Se implementan los recorridos:
 - Inorden
 - Preorden
 - Posorden
5. Se crea la función `mostrarArbol()` para visualizar toda la información almacenada.
6. Se implementa el algoritmo de Dijkstra para calcular caminos mínimos entre nodos considerando el costo de las conexiones.
7. Se desarrollan funciones para leer información desde archivos XML y JSON.

8. Se implementan funciones para exportar la información generada a archivos TXT, CSV, XML y JSON.
9. Finalmente se realizan pruebas para verificar la correcta inserción de nodos, generación de recorridos, cálculo de rutas y almacenamiento de datos.

Comentario

Este programa permite aplicar diversos conceptos fundamentales de estructuras de datos y algoritmos. La implementación del Árbol Binario de Búsqueda facilita la organización eficiente de la información, mientras que la integración del algoritmo de Dijkstra amplía sus capacidades para resolver problemas de rutas mínimas. Además, el manejo de archivos en múltiples formatos mejora la interoperabilidad y el almacenamiento de los datos procesados.

Conclusión

La implementación del Árbol Binario de Búsqueda permitió comprender el funcionamiento de las estructuras jerárquicas y su utilidad para organizar información de manera eficiente. Mediante los recorridos del árbol se puede acceder a los datos desde diferentes perspectivas, mientras que el algoritmo de Dijkstra demuestra cómo estas estructuras pueden utilizarse para resolver problemas reales de navegación y optimización de rutas. Asimismo, la capacidad de importar y exportar información en distintos formatos convierte al sistema en una herramienta flexible, práctica y aplicable a diversos escenarios de gestión de datos.